

```
# CHAPTER-8 : Semantic Searchand Retrieval-AugmentedGeneration
```

```
# ===== Dense retrieval=====
```

```
# Install the latest versions of cohere
!pip install -U cohere
```

```
Requirement already satisfied: cohere in /usr/local/lib/python3.12/dist-packages (7.0.7)
Requirement already satisfied: fastavro<2.0.0,>=1.9.4 in /usr/local/lib/python3.12/dist-packages (from cohere) (1.12.2)
Requirement already satisfied: httpx>=0.25.0 in /usr/local/lib/python3.12/dist-packages (from cohere) (0.28.1)
Requirement already satisfied: pydantic>=1.9.2 in /usr/local/lib/python3.12/dist-packages (from cohere) (2.13.4)
Requirement already satisfied: pydantic-core<3.0.0,>=2.18.2 in /usr/local/lib/python3.12/dist-packages (from cohere) (2.46.4)
Requirement already satisfied: requests<3.0.0,>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from cohere) (2.32.4)
Requirement already satisfied: tokenizers<1,>=0.15 in /usr/local/lib/python3.12/dist-packages (from cohere) (0.22.2)
Requirement already satisfied: types-requests<3.0.0,>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from cohere) (2.33.0)
Requirement already satisfied: typing_extensions>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from cohere) (4.16.0)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx>=0.25.0->cohere) (4.14.2)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx>=0.25.0->cohere) (2026.6.17)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx>=0.25.0->cohere) (1.0.9)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx>=0.25.0->cohere) (3.18)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx>=0.25.0->cohere) (0.14.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic>=1.9.2->cohere) (0.7.0)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic>=1.9.2->cohere) (0.4.2)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.0.0->cohere) (3.4.4)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.0.0->cohere) (2.3.1)
Requirement already satisfied: huggingface-hub<2.0,>=0.16.4 in /usr/local/lib/python3.12/dist-packages (from tokenizers<1,>=0.15->cohere) (0.27.0)
Requirement already satisfied: click<9.0.0,>=8.4.2 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.16.4->cohere) (8.1.8)
Requirement already satisfied: filelock>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.16.4->cohere) (3.10.0)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.16.4->cohere) (2025.10.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.5.1 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.16.4->cohere) (1.5.1)
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.16.4->cohere) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.16.4->cohere) (6.0.2)
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.16.4->cohere) (4.67.1)
```

```
# Uninstall the version you have
!pip uninstall -y cohere
```

```
# used to install the specific version
!pip install cohere==4.57
```

```
Found existing installation: cohere 4.57
Uninstalling cohere-4.57:
  Successfully uninstalled cohere-4.57
Collecting cohere==4.57
  Using cached cohere-4.57-py3-none-any.whl.metadata (6.2 kB)
Requirement already satisfied: aiohttp<4.0,>=3.0 in /usr/local/lib/python3.12/dist-packages (from cohere==4.57) (3.14.1)
Requirement already satisfied: backoff<3.0,>=2.0 in /usr/local/lib/python3.12/dist-packages (from cohere==4.57) (2.2.1)
Requirement already satisfied: fastavro<2.0,>=1.8 in /usr/local/lib/python3.12/dist-packages (from cohere==4.57) (1.12.2)
Requirement already satisfied: importlib_metadata<7.0,>=6.0 in /usr/local/lib/python3.12/dist-packages (from cohere==4.57) (8.5.0)
Requirement already satisfied: requests<3.0.0,>=2.25.0 in /usr/local/lib/python3.12/dist-packages (from cohere==4.57) (2.32.4)
Requirement already satisfied: urllib3<3,>=1.26 in /usr/local/lib/python3.12/dist-packages (from cohere==4.57) (2.5.0)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0,>=3.0->cohere==4.57) (2.5.0)
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0,>=3.0->cohere==4.57) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0,>=3.0->cohere==4.57) (25.4.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0,>=3.0->cohere==4.57) (1.5.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0,>=3.0->cohere==4.57) (6.4.0)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0,>=3.0->cohere==4.57) (0.3.0)
Requirement already satisfied: typing_extensions>=4.4 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0,>=3.0->cohere==4.57) (4.16.0)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0,>=3.0->cohere==4.57) (1.18.3)
Requirement already satisfied: zipp>=0.5 in /usr/local/lib/python3.12/dist-packages (from importlib_metadata<7.0,>=6.0->cohere==4.57) (3.20.2)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.25.0->cohere==4.57) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.25.0->cohere==4.57) (3.18)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.25.0->cohere==4.57) (2026.6.17)
Using cached cohere-4.57-py3-none-any.whl (52 kB)
Installing collected packages: cohere
Successfully installed cohere-4.57
```

```
# Coherent client
```

```
# cohere = it is a python library that allows us to communicate with cohere AI
# It is used to generate embedding
import cohere

# NumPy used for numbers and arrays calculations
import numpy as np

# Pandas library used to work with data in tables
import pandas as pd

# used to display the progress bar
```

```
from tqdm import tqdm

# Paste your API key here. Remember to not share publicly

# api key = it is like a password . It helps to access the cohere ai serives
api_key = 'API_KEY'

# creating cohere client so we can use the cohere ai with our key
# Create and retrieve a Cohere API key from os.cohere.ai
co = cohere.Client(api_key)
```

```
# Text chunking
```

```
# Information like a document
text = """
Interstellar is a 2014 epic science fiction film co-written,
directed, and produced by Christopher Nolan.
It stars Matthew McConaughey, Anne Hathaway, Jessica Chastain,
Bill Irwin, Ellen Burstyn, Matt Damon, and Michael Caine.
Set in a dystopian future where humanity is struggling to
survive, the film follows a group of astronauts who travel
through a wormhole near Saturn in search of a new home for
mankind.
Brothers Christopher and Jonathan Nolan wrote the screenplay,
which had its origins in a script Jonathan developed in 2007.
Caltech theoretical physicist and 2017 Nobel laureate in
Physics[4] Kip Thorne was an executive producer, acted as a
scientific consultant, and wrote a tie-in book, The Science of
Interstellar.
Cinematographer Hoyte van Hoytema shot it on 35 mm movie film in
the Panavision anamorphic format and IMAX 70 mm.
Principal photography began in late 2013 and took place in
Alberta, Iceland, and Los Angeles.
Interstellar uses extensive practical and miniature effects and
the company Double Negative created additional digital effects.
Interstellar premiered on October 26, 2014, in Los Angeles.
In the United States, it was first released on film stock,
expanding to venues using digital projectors.
The film had a worldwide gross over $677 million (and $773
million with subsequent re-releases), making it the tenth-highest
grossing film of 2014.
It received acclaim for its performances, direction, screenplay,
musical score, visual effects, ambition, themes, and emotional
weight.
It has also received praise from many astronomers for its
scientific accuracy and portrayal of theoretical astrophysics.
Since its premiere, Interstellar gained a cult following,[5] and
now is regarded by many sci-fi experts as one of the best
science-fiction films of all time.
Interstellar was nominated for five awards at the 87th Academy
Awards, winning Best Visual Effects, and received numerous other
accolades"""

# Splits the text into sentences where . appears it beraks there
texts = text.split('.')

# for every sentence in texts variable it removes the unwanted spaces and new lines
texts = [t.strip(' \n') for t in texts]
```

```
# Ebedding the text chunks
```

```
# co = previously we created the cohere client
# co.embed = here we are telling that use these things and give me the embeddings
response = co.embed(

    # embed the texts
    texts=texts,

    # use this model for embeddings
    model="embed-english-v3.0",

    # it is a paramater of cohere that the role of text
    # we are telling that this texts will be used later for searching puprose
    input_type="search_document",

).embeddings # cohere retruns the result including embedding , metadata and other info
# this tells to take only embeddings

# convert the embeddings into numpy array
```

```
embeds = np.array(response)

# Shape of the embedding vector
print(embeds.shape)
```

```
(15, 1024)
```

```
# FIASS = Facebook AI Similarity search
# Helps to search the similairty of vectors
!pip install faiss-cpu
```

```
Requirement already satisfied: faiss-cpu in /usr/local/lib/python3.12/dist-packages (1.14.3)
Requirement already satisfied: numpy>=1.25 in /usr/local/lib/python3.12/dist-packages (from faiss-cpu) (2.0.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from faiss-cpu) (26.2)
```

```
# Building the search index
```

```
# Importing FAISS library
import faiss

# in shape we have 2 dimensions
# 0 = No.of.texts
# 1 = No.of.values in embedding vector
dim = embeds.shape[1]

# creating the faiss index to store the values later
# this tells to create the index for the vector which contains dim values in each vector
index = faiss.IndexFlatL2(dim)

# Checking the index is ready or not
print(index.is_trained)

# Faiss accpets the vectors in a specific numerical format(32 bit)
# Convert the embedding vectors into floating array numbers
# add these to faiss index
index.add(np.float32(embeds))
```

```
True
```

```
# Seraching the indexes
```

```
# Function has 2 paramters
# query=users search question
# number_of_results = how many search results does the model want to return
def search(query, number_of_results=3):

    # 1. Get the query's embedding
    query_embed = co.embed(

        # use the list that contains the query for generating embeddings
        texts=[query],

        # model name for using
        model="embed-english-v3.0",

        # Tells this was a search query
        input_type="search_query"

        # take the first embedding vector from returned results
    ).embeddings[0]

    """ 2. Retrieve the nearest neighbors
    search the faiss index using the query and return the required no.of.results
    distances = This contains the distances between the query and the closest vectors.
    similar_item_ids = faiss tells the which vectors are close
    IDs help us find the original text """
    distances, similar_item_ids = index.search(

        # convert the query vector into 32 bit floating point numbers which is expected by faiss
        np.float32([query_embed]),
        number_of_results
    )

    # texts = python list , convert it into array
    # 3. Format the results
    texts_np = np.array(texts)

    # # Create a DataFrame to store the search results
```

```

results = pd.DataFrame(
    data={
        # Get the actual text of the documents found by FAISS using their indexes
        'texts': texts_np[similar_item_ids[0]],

        # Get the distance score for each retrieved document
        'distance': distances[0]
    }
)

# Print the user's search query and a heading for the retrieved results
print(f"Query: '{query}'\n\nNearest neighbors:")

# Return the DataFrame containing the retrieved texts and their distance scores
return results

```

```

query = "how precise was the science"
results = search(query)
print(results)

```

Query: 'how precise was the science'

Nearest neighbors:

	texts	distance
0	It has also received praise from many astronom...	1.200073
1	Caltech theoretical physicist and 2017 Nobel l...	1.370131
2	Interstellar uses extensive practical and mini...	1.580468

```
# ===== Keyword Search by bm25 =====
```

```
!pip install rank-bm25
```

```
Requirement already satisfied: rank-bm25 in /usr/local/lib/python3.12/dist-packages (0.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from rank-bm25) (2.0.2)
```

```

# Import BM25Okapi for keyword-based text ranking
from rank_bm25 import BM25Okapi

# Import the built-in English stop words
from sklearn.feature_extraction import _stop_words

# used for handling punctuation
import string

```

```
# BM25 tokenizer
```

```

# creating a function
def bm25_tokenizer(text):

    # create an empty list
    tokenized_doc = []

    # convert the text into lowercase and split the sentence into individual words
    for token in text.lower().split():

        # removes the punctuation from beginning and end of word
        token = token.strip(string.punctuation)

        # length of token must be > 0 if "?" after removing punctutaion token becomes empty
        # checking the token is stop word or not
        if len(token) > 0 and token not in _stop_words.ENGLISH_STOP_WORDS:

            # if not a stop word then add the token to our tokenized_doc list
            tokenized_doc.append(token)

    # return the list
    return tokenized_doc

```

```
# Document (Information) tokenizer
```

```

from tqdm import tqdm
# stores the cleaned texts / doc
tokenized_corpus = []

# go through the each sentence in the texts/doc remove stop words and add cleaned texts to tokenized_corpus = []

```

```
for passage in tqdm(texts):
    tokenized_corpus.append(bm25_tokenizer(passage))
```

```
100%|██████████| 15/15 [00:00<00:00, 19082.37it/s]
```

```
# BM25 model
```

```
# creating bm25 object to create
# bm25 search system using the tokenized documents
bm25 = BM25Okapi(tokenized_corpus)
```

```
# Search function
```

```
import numpy as np

# top_k = how many results you want to display
# num_candidates = how many sentences you want to consider for search
def keyword_search(query, top_k=3, num_candidates=15):
    print("Input question:", query)

    # we have cleaned list query in bm25 we have cleaned list of documents
    # check how relevant every document to the query and return the score
    # high score = more relevant
    bm25_scores = bm25.get_scores(
        bm25_tokenizer(query)
    )

    # Get top candidate document IDs
    # np.argpartition = helps to find the indexes of the high score documents
    # np.argpartition() rearranges the array so that the top 3 candidate indexes are placed at the end (not sorted)
    top_n = np.argpartition(

        # we have score of every document
        bm25_scores,

        # this tells to take the n top results
        -num_candidates
    )[-num_candidates:] # take the last 3 positions cause it have high score doc

    # This having selected document's indexes and scores
    bm25_hits = [
        {
            'corpus_id': idx, # stores the id of doc
            'score': bm25_scores[idx] # stores the score
        }
        for idx in top_n # Goes through each document index stored in top_n
    ]

    # Sort results by score (highest first)
    bm25_hits = sorted(

        # we have top selected doc with index and scores
        bm25_hits,

        # x= one document , Gets the score value from that item
        key=lambda x: x['score'],

        # sort in descending order
        reverse=True
    )

    print("Top-3 lexical search (BM25) hits")

    # Print top-k results
    # Takes the first top_k results from the sorted bm25_hits list
    for hit in bm25_hits[0:top_k]:
        print(

            # t → adds a tab space
            # {:.3f} → displays the BM25 score with 3 decimal places
            # {} → displays the document text
            # .format(...) → puts the actual values into {:.3f} and {}
            "\t{:.3f}\t{}".format(

                # Gets the BM25 score of the current result
                hit['score'],

                # Uses the document's ID/index to get the original document text.
                # Replaces new-line characters with spaces so the document is printed in one line
```

```

        texts[hit['corpus_id']].replace("\n", " ")
    )
)

```

```

# call the function
keyword_search(query = "how precise was the science")

```

Input question: how precise was the science

Top-3 lexical search (BM25) hits

```

1.789 Interstellar is a 2014 epic science fiction film co-written, directed, and produced by Christopher Nolan
1.373 Caltech theoretical physicist and 2017 Nobel laureate in Physics[4] Kip Thorne was an executive producer, ac
0.000 It stars Matthew McConaughey, Anne Hathaway, Jessica Chastain, Bill Irwin, Ellen Burstyn, Matt Damon, and Mi

```

RERANKING

```

query = "how precise was the science"
# The reranker model directly evaluates and ranks the documents
# Send the query and all documents to Cohere's reranking model
results = co.rerank(query=query, documents=texts, top_n=3,

```

```

    # Include the actual document text in the returned results
    return_documents=True
)

```

```

# Get the list of reranked results from the response
results.results

```

```

[RerankResponseResultsItem(document=RerankResponseResultsItemDocument(text='It has also received praise from many
astronomers for its\nscientific accuracy and portrayal of theoretical astrophysics'), index=12,
relevance_score=0.15232232),
 RerankResponseResultsItem(document=RerankResponseResultsItemDocument(text='The film had a worldwide gross over $677
million (and $773\nmillion with subsequent re-releases), making it the tenth-highest\ngrossing film of 2014'), index=10,
relevance_score=0.050354082),
 RerankResponseResultsItem(document=RerankResponseResultsItemDocument(text='Interstellar is a 2014 epic science fiction
film co-written,\ndirected, and produced by Christopher Nolan'), index=0, relevance_score=0.0350424)]

```

```

# enumerate() gives us two things for each result:
# idx = the position/index of the result
# result = the actual result object

# Go through each reranked result and print its position, relevance score, and actual document text.
for idx, result in enumerate(results.results):
    print(idx, result.relevance_score , result.document.text)

```

```

0 0.15232232 It has also received praise from many astronomers for its
scientific accuracy and portrayal of theoretical astrophysics
1 0.050354082 The film had a worldwide gross over $677 million (and $773
million with subsequent re-releases), making it the tenth-highest
grossing film of 2014
2 0.0350424 Interstellar is a 2014 epic science fiction film co-written,
directed, and produced by Christopher Nolan

```

```

# KEYWORD AND RERANKING
# BM25 also produces a top 3 for display, but those are not the final results.
# The reranker takes the 10 BM25 candidates and chooses a new final top 3.

```

```

def keyword_and_reranking_search(query, top_k=3, num_candidates=10):
    print("Input question:", query)

    # 1. BM25 search
    bm25_scores = bm25.get_scores(
        bm25_tokenizer(query)
    )

    # 2. Get top candidate document IDs
    top_n = np.argpartition(
        bm25_scores,
        -num_candidates
    )[-num_candidates:]

    # 3. Create BM25 hits
    bm25_hits = [
        {
            'corpus_id': idx,
            'score': bm25_scores[idx]
        }
        for idx in top_n
    ]

```

```

# 4. Sort BM25 results
bm25_hits = sorted(
    bm25_hits,
    key=lambda x: x['score'],
    reverse=True
)

# 5. Print top BM25 results
print("Top-3 lexical search (BM25) hits")

for hit in bm25_hits[0:top_k]:
    print(
        "\t{:.3f}\t{}".format(
            hit['score'],
            texts[hit['corpus_id']].replace("\n", " ")
        )
    )

# 6. Get documents for reranking
# Extract the original text of all BM25 candidate documents
docs = [
    texts[hit['corpus_id']]
    for hit in bm25_hits
]

# 7. Rerank using Cohere
# Print a heading showing that BM25 results will now be reranked
print(
    f"\n\nTop-{top_k} hits by rank-API "
    f"({len(bm25_hits)} BM25 hits re-ranked)"
)

# Send the query and BM25 candidate documents to Cohere's reranker
results = co.rerank(
    query=query,
    documents=docs,
    top_n=top_k,
    return_documents=True
)

# 8. Print reranked results
# Go through each result returned by the Cohere reranker
for hit in results.results:
    print(

        # Print the rerank relevance score and the corresponding document text
        "\t{:.3f}\t{}".format(
            hit.relevance_score,
            hit.document.text.replace("\n", " ")
        )
    )

```

```
keyword_and_reranking_search(query = "how precise was the science")
```

Input question: how precise was the science

Top-3 lexical search (BM25) hits

```

1.789  Interstellar is a 2014 epic science fiction film co-written, directed, and produced by Christopher Nolan
1.373  Caltech theoretical physicist and 2017 Nobel laureate in Physics[4] Kip Thorne was an executive producer, ac
0.000  Principal photography began in late 2013 and took place in Alberta, Iceland, and Los Angeles

```

Top-3 hits by rank-API (10 BM25 hits re-ranked)

```

0.152  It has also received praise from many astronomers for its scientific accuracy and portrayal of theoretical a
0.050  The film had a worldwide gross over $677 million (and $773 million with subsequent re-releases), making it t
0.035  Interstellar is a 2014 epic science fiction film co-written, directed, and produced by Christopher Nolan

```

```
# ===== Retrieval-Augmented Generation (RAG) =====
```

```
query = "income generated"
```

```
# 1- Retrieval
```

```
# We'll use embedding search. But ideally we'd do hybrid
```

```
# Search the document collection using embedding search
```

```
# The search function returns the documents most relevant to the query
```

```
results = search(query)
```

```
# 2- Grounded Generation
```

```
# Convert the retrieved document texts into dictionaries
```

```
# because the co.chat() function expects documents in this format
```

```
docs_dict = [{'text': text}
```

```
for text in results['texts']]
```

The film had a worldwide gross of over \$677 million (and \$773 million with subsequent re-releases), making it the tenth-high

2026-07-24 04:19:42 (73.3 MB/s) - 'Phi-3-mini-4k-instruct-fp16.gguf.4' saved [7643295904/7643295904]

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
Loading weights: 100% 199/199 [00:00<00:00, 4130.03it/s]
```

```
{context}
Provide a concise answer the following question using the
relevant information provided above:
{question}<|end|>
<|assistant|>""

# Create a PromptTemplate object using the template above
prompt = PromptTemplate(
    template=template,
    input_variables=["context", "question"]
)
```

```
!pip uninstall -y langchain langchain-core langchain-community
!pip install langchain==0.0.353
!pip install llama-cpp-python
```

```
sting installation: langchain 0.0.353
ing langchain-0.0.353:
fully uninstalled langchain-0.0.353
sting installation: langchain-core 0.1.23
ing langchain-core-0.1.23:
fully uninstalled langchain-core-0.1.23
sting installation: langchain-community 0.0.20
ing langchain-community-0.0.20:
fully uninstalled langchain-community-0.0.20
g langchain==0.0.353
ached langchain-0.0.353-py3-none-any.whl.metadata (13 kB)
nt already satisfied: PyYAML>=5.3 in /usr/local/lib/python3.12/dist-packages (from langchain==0.0.353) (6.0.3)
nt already satisfied: SQLAlchemy<3,>=1.4 in /usr/local/lib/python3.12/dist-packages (from langchain==0.0.353) (2.0.51)
nt already satisfied: aiohttp<4.0.0,>=3.8.3 in /usr/local/lib/python3.12/dist-packages (from langchain==0.0.353) (3.14.1)
nt already satisfied: dataclasses-json<0.7,>=0.5.7 in /usr/local/lib/python3.12/dist-packages (from langchain==0.0.353) (0.
nt already satisfied: jsonpatch<2.0,>=1.33 in /usr/local/lib/python3.12/dist-packages (from langchain==0.0.353) (1.33)
g langchain-community<0.1,>=0.0.2 (from langchain==0.0.353)
ached langchain_community-0.0.38-py3-none-any.whl.metadata (8.7 kB)
g langchain-core<0.2,>=0.1.4 (from langchain==0.0.353)
ached langchain_core-0.1.53-py3-none-any.whl.metadata (5.9 kB)
nt already satisfied: langsmith<0.1.0,>=0.0.70 in /usr/local/lib/python3.12/dist-packages (from langchain==0.0.353) (0.0.87
nt already satisfied: numpy<2,>=1 in /usr/local/lib/python3.12/dist-packages (from langchain==0.0.353) (1.26.4)
nt already satisfied: pydantic<3,>=1 in /usr/local/lib/python3.12/dist-packages (from langchain==0.0.353) (2.13.4)
nt already satisfied: requests<3,>=2 in /usr/local/lib/python3.12/dist-packages (from langchain==0.0.353) (2.34.2)
nt already satisfied: tenacity<9.0.0,>=8.1.0 in /usr/local/lib/python3.12/dist-packages (from langchain==0.0.353) (8.5.0)
nt already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0.0,>=3.8.3->langc
nt already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0.0,>=3.8.3->langchain==0
nt already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0.0,>=3.8.3->langchain==0.0
nt already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0.0,>=3.8.3->langchain==
nt already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0.0,>=3.8.3->langchain
nt already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0.0,>=3.8.3->langchain==0
nt already satisfied: typing_extensions>=4.4 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0.0,>=3.8.3->langch
nt already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0.0,>=3.8.3->langchain==
nt already satisfied: marshmallow<4.0.0,>=3.18.0 in /usr/local/lib/python3.12/dist-packages (from dataclasses-json<0.7,>=0.
nt already satisfied: typing-inspect<1,>=0.4.0 in /usr/local/lib/python3.12/dist-packages (from dataclasses-json<0.7,>=0.5
nt already satisfied: jsonpointer>=1.9 in /usr/local/lib/python3.12/dist-packages (from jsonpatch<2.0,>=1.33->langchain==0.
is looking at multiple versions of langchain-community to determine which version is compatible with other requirements. T
g langchain-community<0.1,>=0.0.2 (from langchain==0.0.353)
ached langchain_community-0.0.37-py3-none-any.whl.metadata (8.7 kB)
ached langchain_community-0.0.36-py3-none-any.whl.metadata (8.7 kB)
ached langchain_community-0.0.35-py3-none-any.whl.metadata (8.7 kB)
ached langchain_community-0.0.34-py3-none-any.whl.metadata (8.5 kB)
ached langchain_community-0.0.33-py3-none-any.whl.metadata (8.5 kB)
ached langchain_community-0.0.32-py3-none-any.whl.metadata (8.5 kB)
ached langchain_community-0.0.31-py3-none-any.whl.metadata (8.4 kB)
is still looking at multiple versions of langchain-community to determine which version is compatible with other requireme
ached langchain_community-0.0.30-py3-none-any.whl.metadata (8.4 kB)
ached langchain_community-0.0.29-py3-none-any.whl.metadata (8.3 kB)
ached langchain_community-0.0.28-py3-none-any.whl.metadata (8.3 kB)
ached langchain_community-0.0.27-py3-none-any.whl.metadata (8.2 kB)
ached langchain_community-0.0.26-py3-none-any.whl.metadata (8.2 kB)
s is taking longer than usual. You might need to provide the dependency resolver with stricter constraints to reduce runtim
ached langchain_community-0.0.25-py3-none-any.whl.metadata (8.1 kB)
ached langchain_community-0.0.24-py3-none-any.whl.metadata (8.1 kB)
ached langchain_community-0.0.23-py3-none-any.whl.metadata (8.1 kB)
ached langchain_community-0.0.22-py3-none-any.whl.metadata (8.1 kB)
ached langchain_community-0.0.21-py3-none-any.whl.metadata (8.1 kB)
```

```
# Import RetrievalQA, which helps create a RAG question-answering pipeline
from langchain.chains import RetrievalQA
```

```
# RAG pipeline
rag = RetrievalQA.from_chain_type(
    llm=llm,

    # "stuff" means all retrieved documents are placed together
    # into the prompt as context before sending them to the LLM
    chain_type='stuff',
```

```
# Convert the FAISS database into a retriever
# The retriever searches the vector database and finds
# documents that are relevant to the user's question
retriever=db.as_retriever(),

chain_type_kwargs={
    "prompt": prompt
},
    # Show the intermediate steps and information while the RAG pipeline runs
verbose=True
)
```

Question ==> Retrieve relevant documents ==> Add them to prompt ==> LLM generates answer.
rag.invoke('Income generated')

> Entering new RetrievalQA chain...

> Finished chain.

```
{'query': 'Income generated',
 'result': ' Interstellar generated over $677 million worldwide in its initial release, making it the tenth-highest grossing film of 2014. With subsequent re-releases, this amount increased to approximately $773 million. The movie was shot using traditional methods including 35 mm film and IMAX 70 mm projection, before being released on digital projectors in some venues within the United States.'}
```